Fase 3 — Cliente de sincronização no PDV

O PDV passa a enviar a fila local para a API já publicada, só quando o operador gravar endereço, loja, terminal e segredo. Sem isso, a venda continua só no SQLite.

A API e o MySQL da Fase 2 permanecem. Esta fase só ensina o PDV a esvaziar a fila que já existe. Sem endereço da API e sem segredo, nenhuma tela chama a rede. Login, caixa, catálogo, venda, estoque, ticket e fechamento continuam terminando no SQLite.

O indicador ONLINE / OFFLINE / SINCRONIZANDO / ERRO fica de fora. É o §38.

O id da inscrição não é o id do SQLite

A inscrição feita no Postman devolveu `lojaId`, `terminalId` e `segredo` criados na API. A venda local carrega o `StoreId` e o `TerminalId` do SQLite. O `POST /api/lotes` recusa o corpo quando esses campos não são os do token (`Lote de outra loja` em `src/AcaiPos.Api/Services/LoteService.cs`).

O `Id` da venda, do caixa e dos movimentos continua o do SQLite: é a chave de idempotência. Na hora de montar o JSON, só `StoreId` e `TerminalId` do payload são trocados pelos ids gravados na inscrição. O banco local não é reescrito.

Tela de inscrição

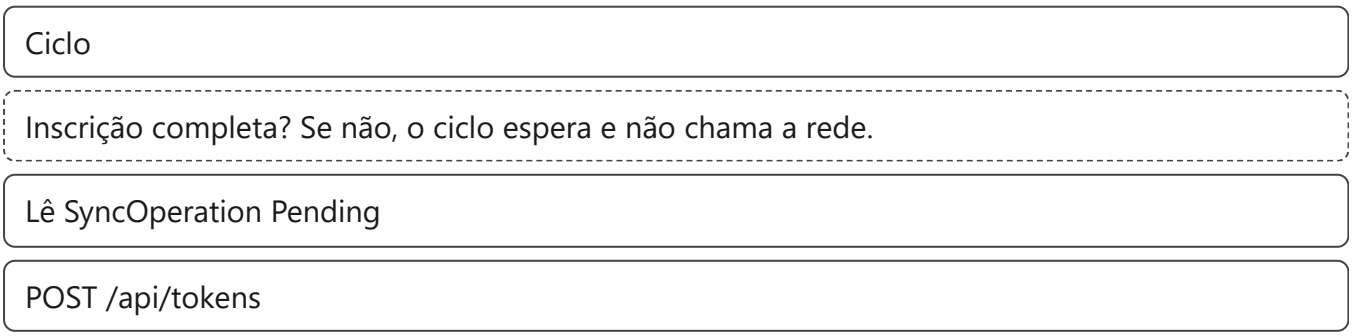
Nova seção na tela de configurações já aberta em `src/AcaiPos.Desktop/MainWindow.xaml`, ao lado de Interface humana. Campos: endereço da API, `lojaId`, `terminalId` e segredo. O segredo é digitado uma vez, protegido com o DPAPI já usado em `src/AcaiPos.Desktop/Fiscal/DpapiProtecaoCertificado.cs` e não volta para a tela em claro.

A inscrição local é uma linha nova, fora de `SyncOperation`. Não entra no lote. `Configuracao` não serve: ela é auditável e `MarkUpdated` a marcaria como pendente. A coluna nova entra por um `Migrate*` em `src/AcaiPos.Infrastructure/DependencyInjection.cs`, no mesmo estilo do schema local. SQLite antigo de teste precisa ser recriado se não tiver a tabela; não há backfill.

Salvar a tela não reinicia o PDV. O ciclo seguinte passa a enxergar a inscrição.

Ciclo

Um `BackgroundService` no host que `src/AcaiPos.Desktop/App.xaml.cs` já sobe. Intervalo de 30 segundos. A tela de venda não espera esse ciclo.



POST /api/lotes de um agregado

200: marca Synced

Rede, 5xx ou 401 após renovar: continua Pending e grava SyncError

400: marca Error

Cada `SyncOperation` pendente vai em um POST próprio. A API recusa o lote inteiro quando um item é inválido; um POST por agregado impede que uma venda ruim segure o caixa.

- 200: a operação e o agregado ficam `Synced`.
- Rede, timeout ou 5xx: continua `Pending`, soma tentativa e grava `SyncError`.
- 401: renova o token uma vez. Se repetir, conta como falha de tentativa.
- 400: `Error`, sem reenviar o mesmo corpo. Uma alteração nova do agregado chama `MarkPending` de novo.

O limite é 5 tentativas. Esgotado, a operação fica `Error` até a próxima alteração local. `MarkPending` em `src/AcaiPos.Domain/Entities/SyncOperation.cs` hoje não zera `AttemptCount` nem oferece `Synced` ou falha. Ganha esses métodos, e a nova alteração zera o contador.

O PDV não referencia o projeto `AcaiPos.Api`. O JSON segue o contrato já publicado em `src/AcaiPos.Api/Dtos/LoteDtos.cs`: venda com itens e pagamentos, caixa, movimento de caixa e movimento de estoque. Venda aberta não está na fila e não é enviada. Catálogo, PIN, NFC-e e o segredo não entram no corpo.

Prova

Testes de domínio: cinco falhas deixam `Error`; uma alteração seguinte volta a `Pending` com contador zero; 400 não incrementa para novo envio cego. Um teste do montador do lote mostra o `StoreId` e o `TerminalId` da inscrição no JSON e o `Id` da venda igual ao do SQLite. A suíte não precisa do MySQL da SaveInCloud.

Entregas previstas: estado da fila, inscrição local, tela, ciclo em background e testes.